

Linear Algebra Made Easy and Visual with Python
Iris Series · Math Made Easy and Visual — by Dr. Jiang

01 Vectors

1.2 Coordinate Systems

Let us visualize vectors in coordinate systems



Linear Algebra Made Easy and Visual | Open-source code & drafts: <https://github.com/Visualize-ML>

Skills You Will Master in This Section

1

Understand the real number line as one-dimensional Euclidean space — origin, positive direction, and unit length

3

Learn to create 2D and 3D grid data with `numpy.meshgrid()`, a foundation for plotting and vectorized computation

5

Build early intuition for “span” and the construction of vector spaces, using unit vectors

2

Master the planar Cartesian system, and how to visualize 2D/3D vectors under the right-hand-rule convention

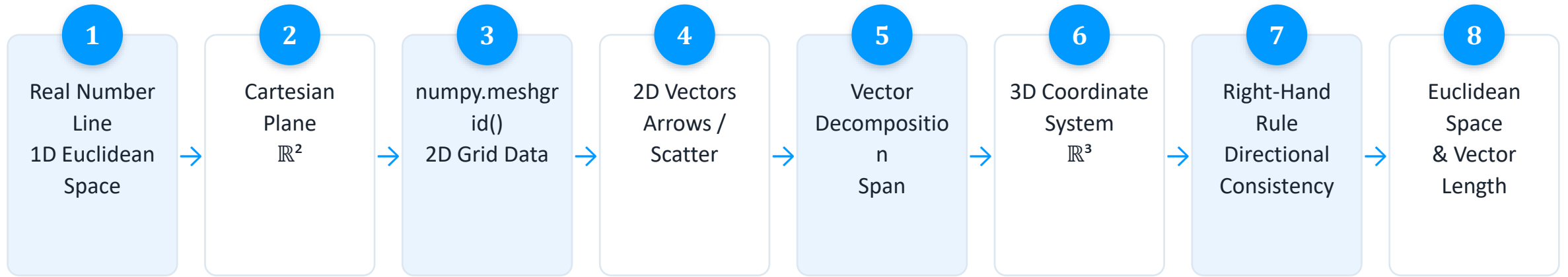
4

Represent and visualize 2D vectors within the planar Cartesian coordinate system

6

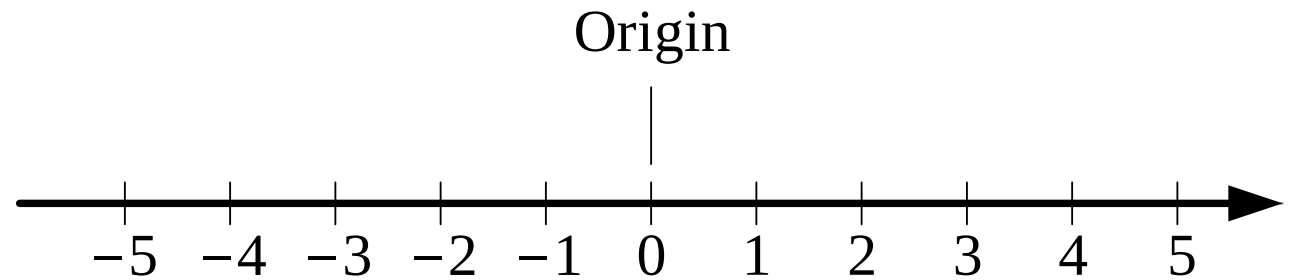
Compute vector length (magnitude) and verify it using the Pythagorean theorem

From a Single Axis to Three-Dimensional Space



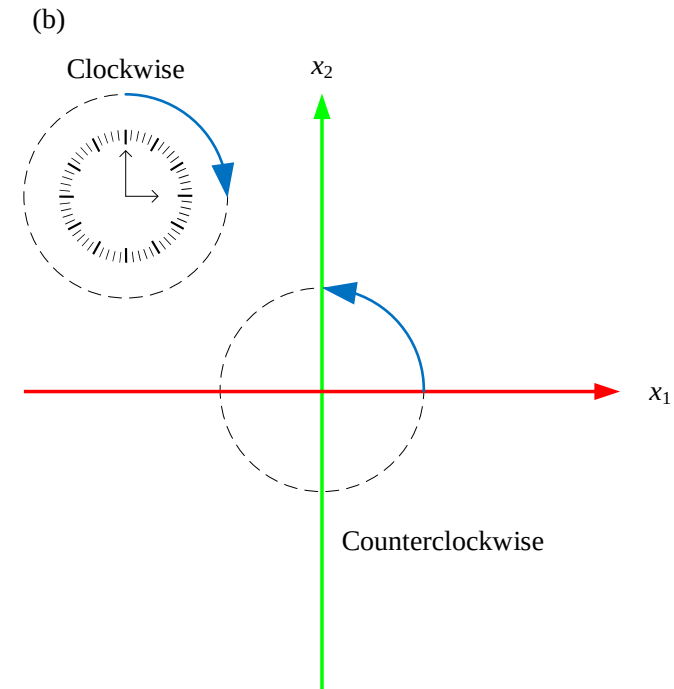
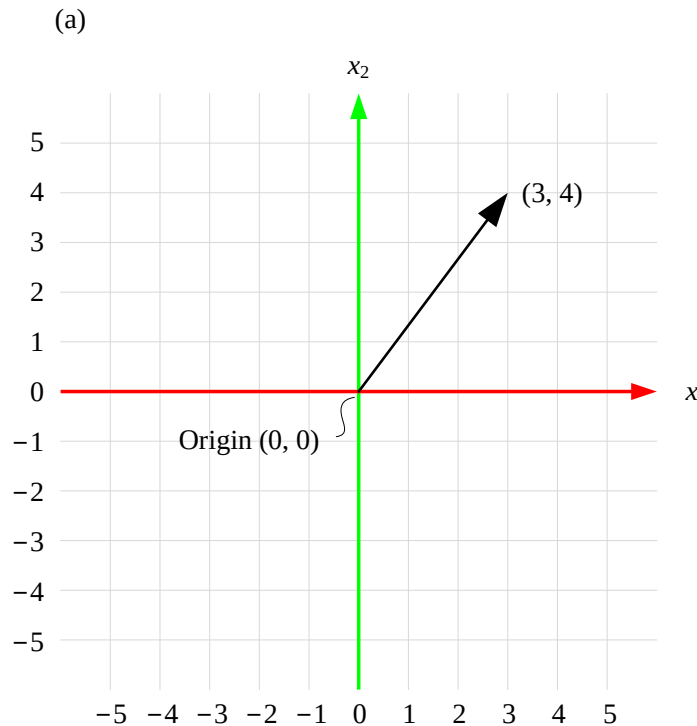
One-Dimensional Euclidean Space

- The real line (real axis) is an infinitely extending line used to locate real numbers (scalars).
- Three key elements: origin, positive direction (arrowhead), and unit length (1).
- Negative numbers lie left of the origin, positive numbers to the right; it extends toward $-\infty$ and $+\infty$.
- The real line is also called one-dimensional Euclidean space, denoted $\mathbb{R}$ — the set of real numbers.



Descartes' Coordinates & 2D Euclidean Space

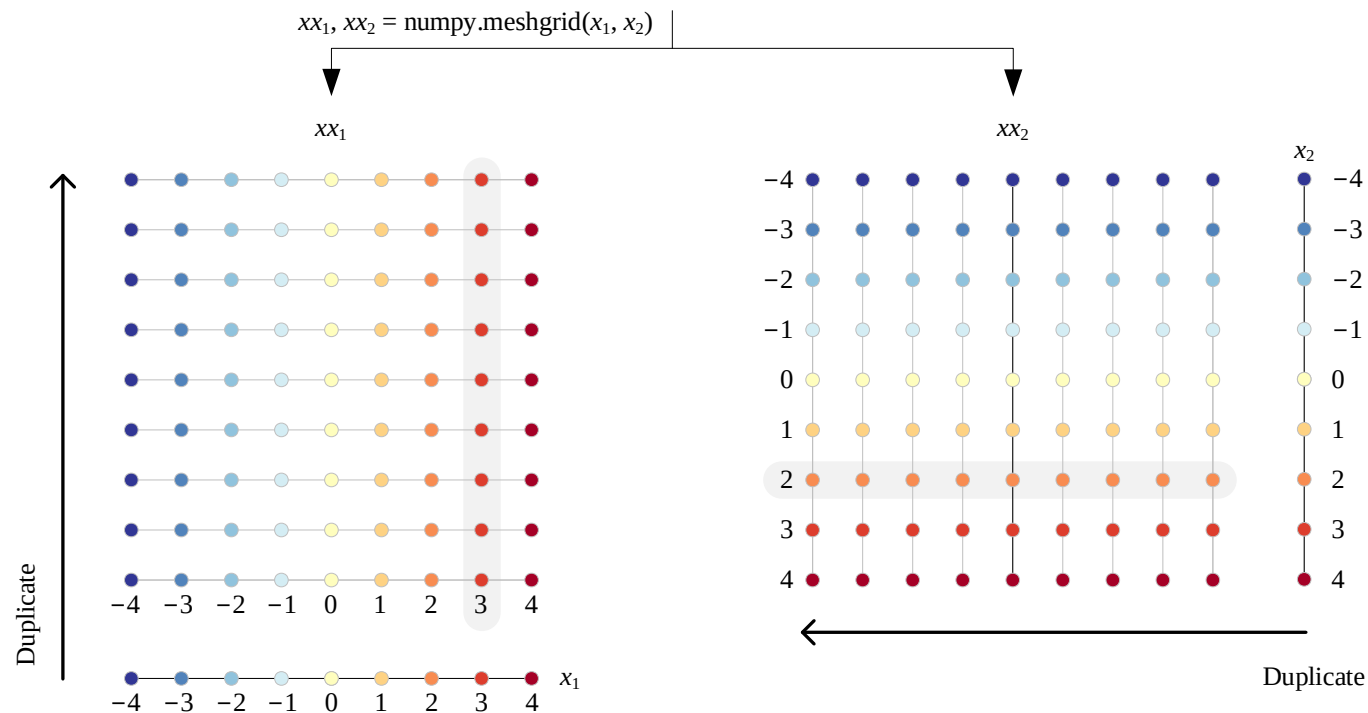
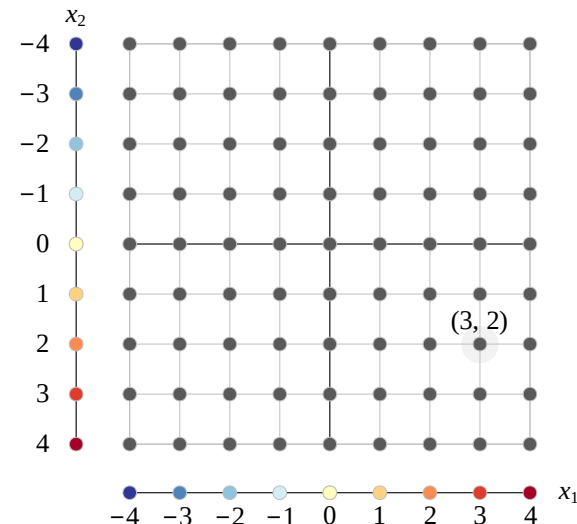
- René Descartes introduced coordinates in the 17th century, fusing geometry with algebra.
- Two perpendicular axes meeting at the origin form the plane — two-dimensional Euclidean space $\mathbb{R}^2$.
- A point is written (x_1, x_2) ; the column vector $\mathbf{x} = [x_1; x_2]$ is an arrow from the origin to that point.
- The x_1 -axis is colored red, the x_2 -axis green — foreshadowing the “red-green plane” used for RGB later.
- Panel (b): counterclockwise is defined positive, matching the right-hand-system standard for determinants, cross products, and rotation.



CREATING 2D GRID DATA

Building a Coordinate Grid with `numpy.meshgrid()`

- `meshgrid(x1, x2)` takes two 1D arrays and returns two 2D grids, `xx1` and `xx2`, for plotting and vectorized math.
- Note: under array-index convention, `xx2` has negatives on top and positives at the bottom — the opposite of typical math axes.



Generating 2D Grid Data

Code 1. Creating a 2D Grid with
`numpy.meshgrid()` | `LA_01_02_01.ipynb`

- a Import NumPy as np
- b `np.arange(-4, 5, 1)` generates 1D arrays for x_1, x_2 (stop value 5 excluded)
- c `np.meshgrid(x1, x2)` produces the 2D grids `xx1, xx2`

a `import numpy as np`

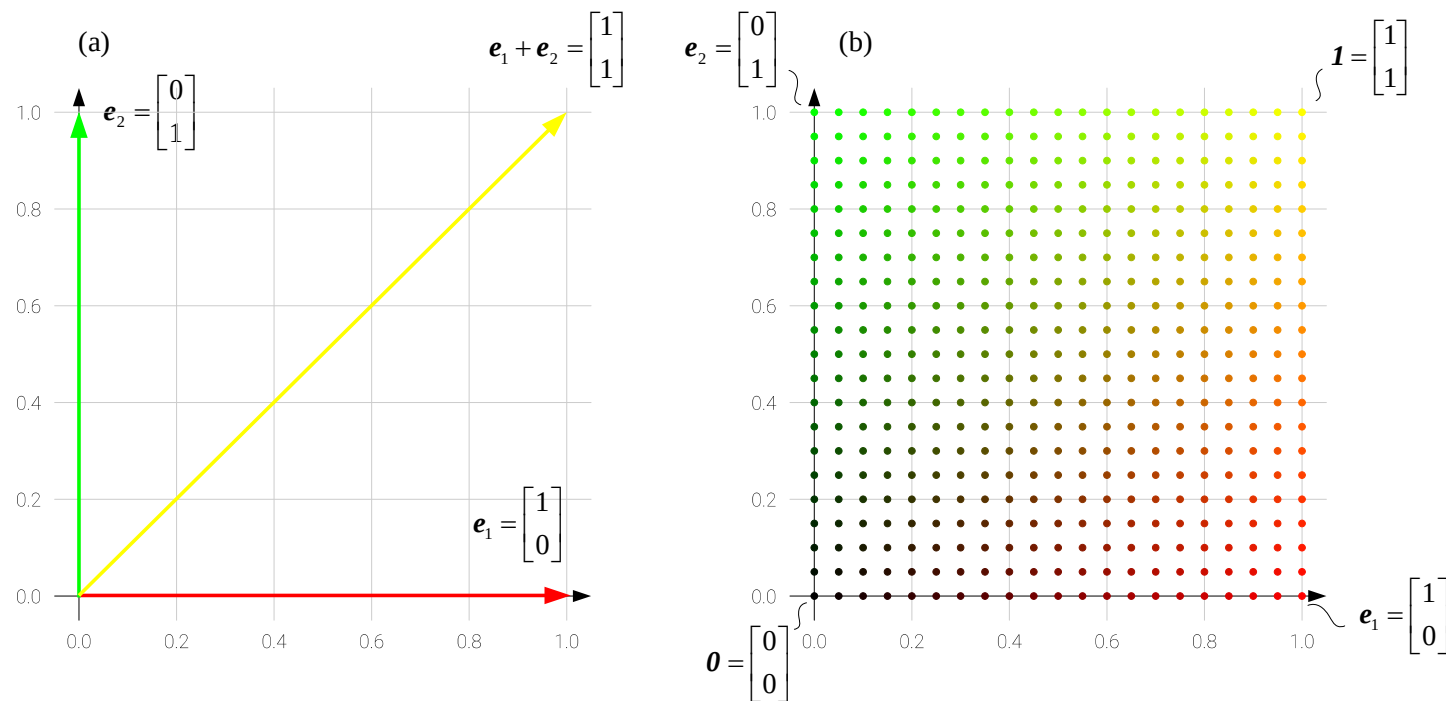
b `# 生成  $x_1$  和  $x_2$  的一维数组, 范围  $[-4, 4]$ , 步长为 1`
`x1 = np.arange(-4, 5, 1)`
`x2 = np.arange(-4, 5, 1)`

c `# 生成网格`
`xx1, xx2 = np.meshgrid(x1, x2)`



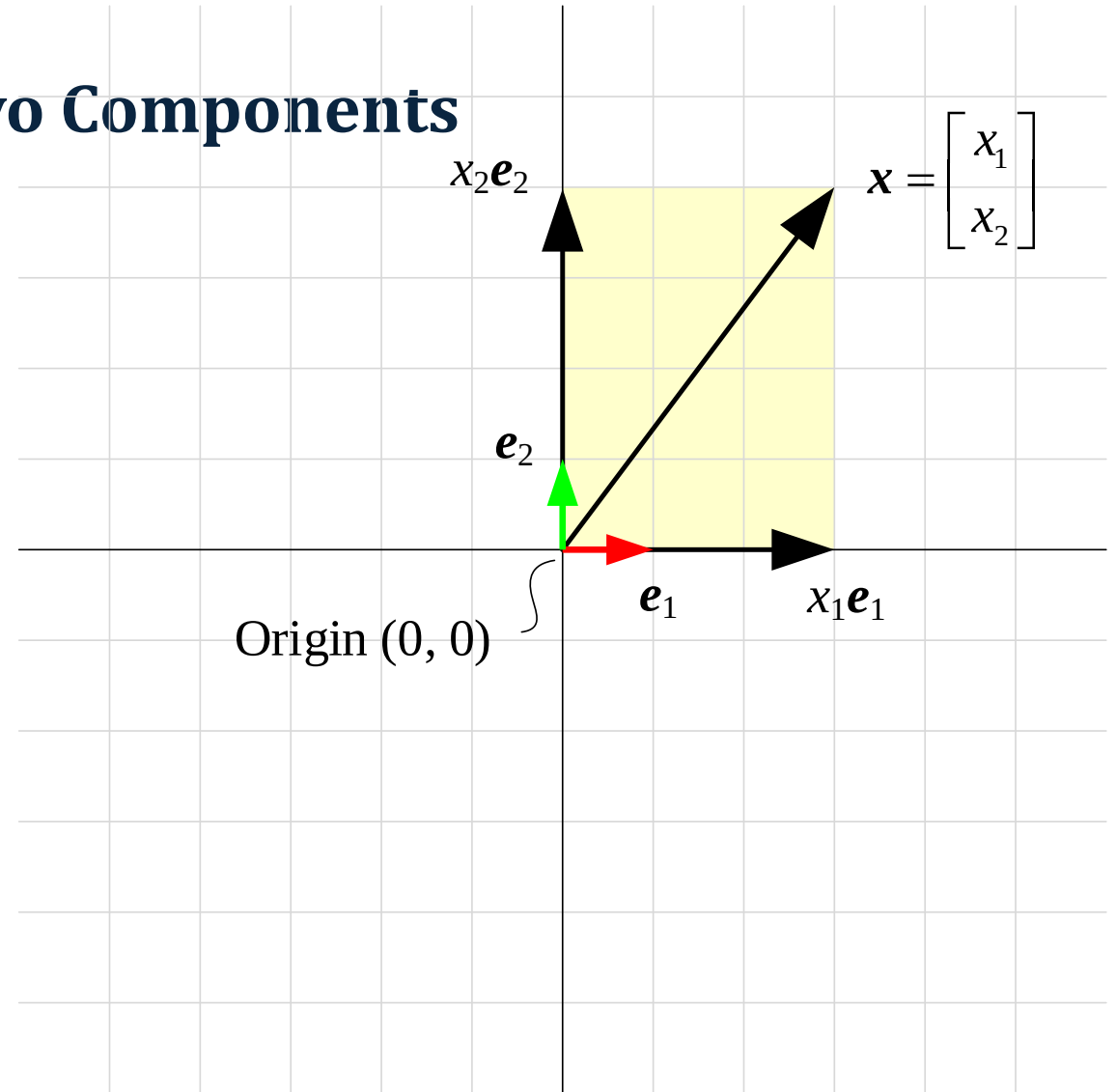
Unit Vectors and Regular Scatter on the “Red-Green” Plane

- Few vectors, exact start/end points matter → use arrows (quiver).
- Many vectors, distribution/trend matters → use a scatter plot; the start point defaults to the origin.
- $\mathbf{e}_1 = [1, 0]^T$ red arrow = pure red light; $\mathbf{e}_2 = [0, 1]^T$ green arrow = pure green light.
- $\mathbf{e}_1 + \mathbf{e}_2 = [1, 1]^T$ yellow arrow = red + green mixed light; the origin $[0, 0]^T$ represents pure black.



Writing a 2D Vector as a Sum of Two Components

- $\mathbf{e}_1 = [1, 0]^T$ and $\mathbf{e}_2 = [0, 1]^T$ are two special vectors on the red-green plane, each of length 1.
- A vector of length 1 is called a unit vector.
- Any vector $\mathbf{x} = [x_1, x_2]^T$ in the plane can be written as $x_1\mathbf{e}_1$ placed head-to-tail with $x_2\mathbf{e}_2$.
- The grid woven by $\mathbf{e}_1$ and $\mathbf{e}_2$ seems to “support” the whole plane — an intuitive picture of span.



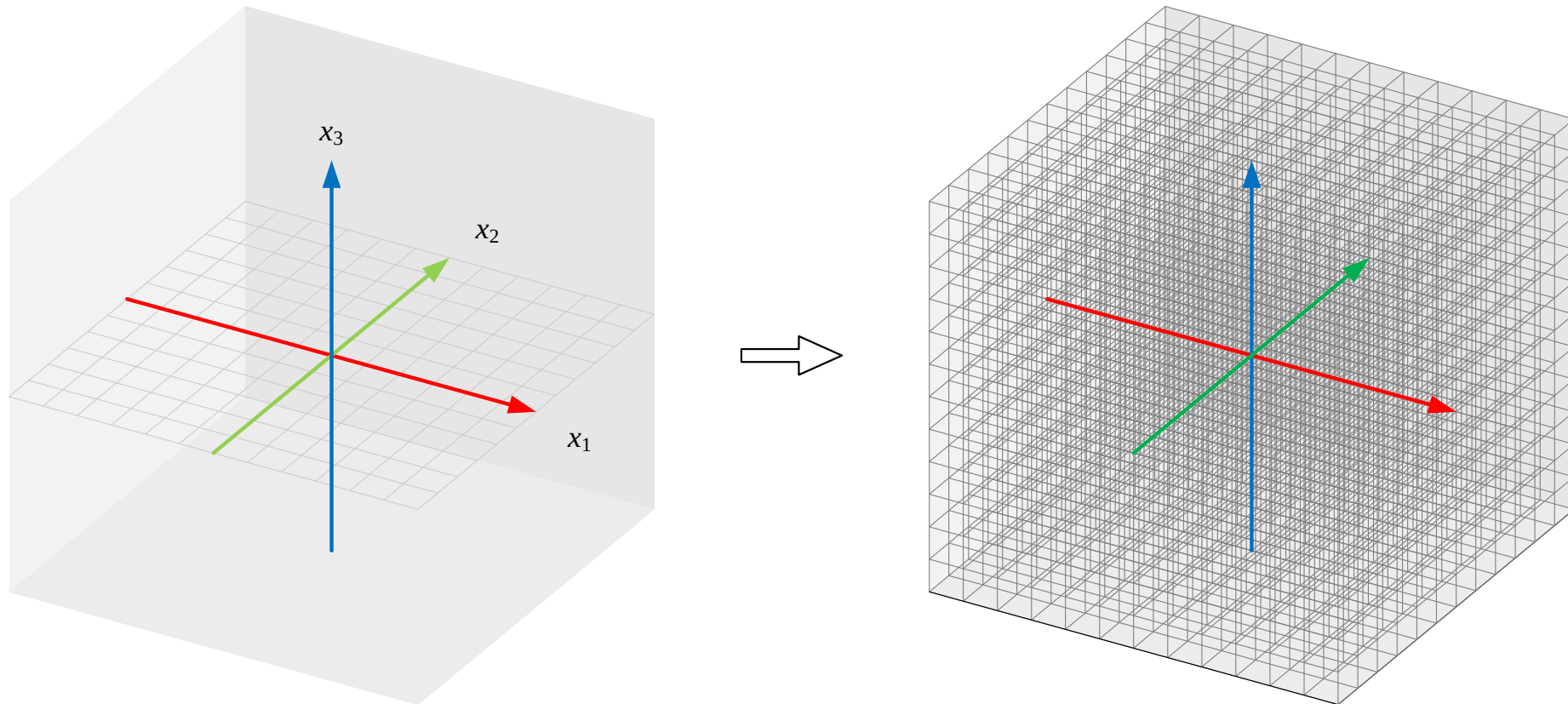
Vector Decomposition

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = x_1 \begin{bmatrix} 1 \\ 0 \end{bmatrix} + x_2 \begin{bmatrix} 0 \\ 1 \end{bmatrix} = x_1 \mathbf{e}_1 + x_2 \mathbf{e}_2$$

- Span: combining a set of vectors linearly to “cover” an entire space — a cornerstone idea of vector spaces.
- This book will revisit and deepen “span” and “vector space” many times going forward.
- Note: because RGB values are bounded to $[0,1]$, the RGB space is not strictly a vector space, but it still builds useful intuition.

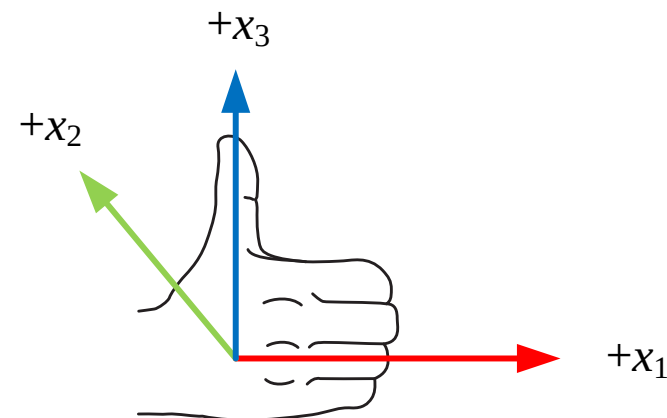
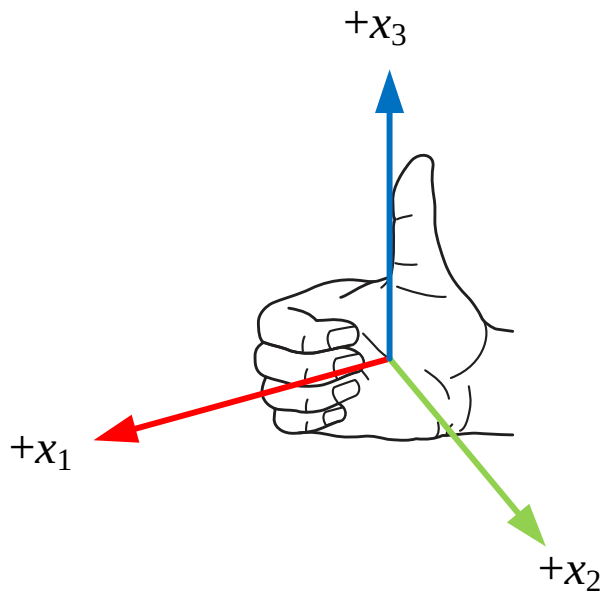
Extending the Plane into Three Dimensions

- Raise a new axis x_3 perpendicular to the x_1x_2 plane at the origin; all three axes are mutually perpendicular $\rightarrow$ the 3D system $\mathbb{R}^3$.
- A point is written (x_1, x_2, x_3) ; the vector $\mathbf{x} = [x_1, x_2, x_3]^T$ is an arrow from the origin to that point.



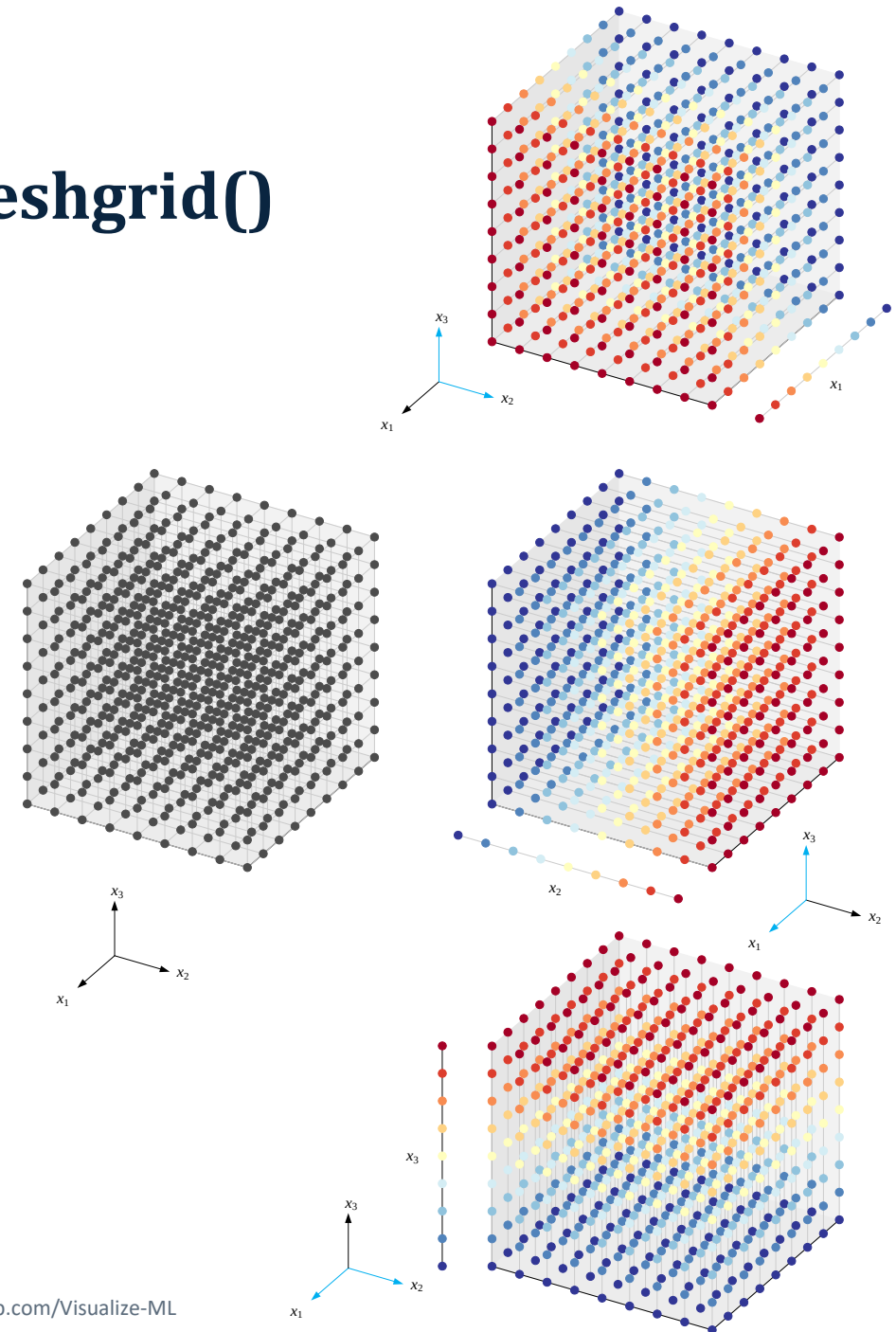
The Right-Hand 3D Coordinate System

- Curl the right hand's fingers from the $+x_1$ axis toward the $+x_2$ axis; the extended thumb then points along $+x_3$.
- The right-hand rule keeps determinants, coordinate transforms, cross products, and rotation direction all consistent.
- Code file LA_01_02_02.ipynb demonstrates generating 3D grid data with `numpy.meshgrid()` — recommended self-study.



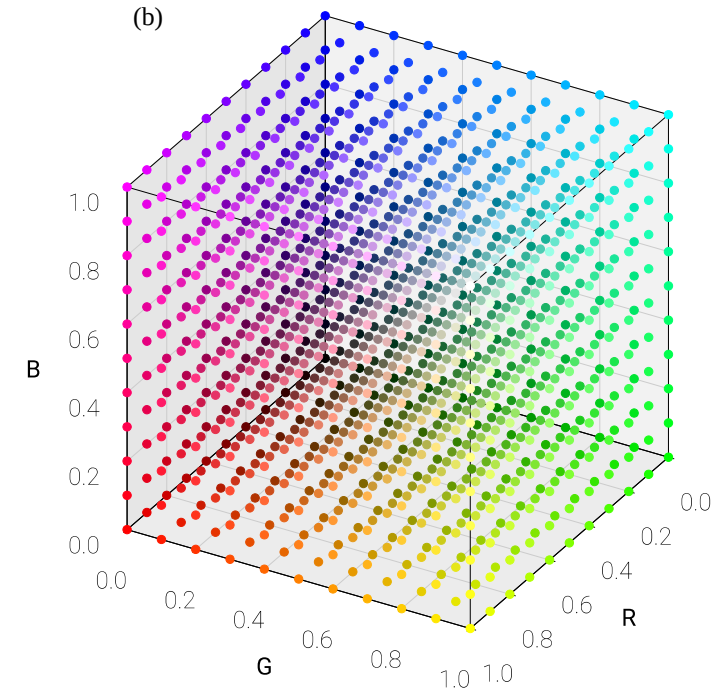
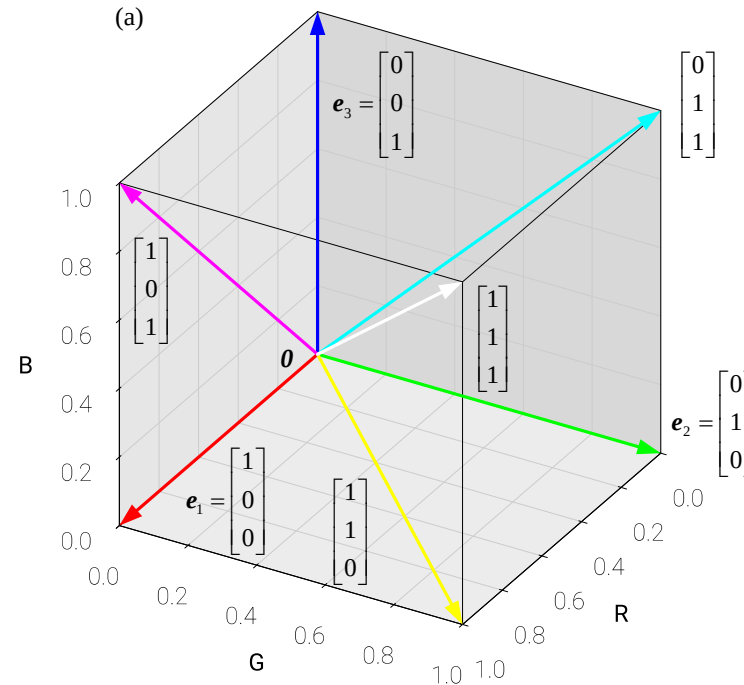
Creating 3D Grid Data with `numpy.meshgrid()`

- A 3D grid is built by “duplicating” three 1D arrays along three directions; blue arrows mark the duplication direction.
- The principle is identical to the 2D case — just one more dimension and one more duplication step.



Arrows and Scatter in RGB Color Space

- The red-green plane is only a thin slice of the full RGB color universe; adding blue unlocks the full palette.
- $\mathbf{e}_1$ = pure red, $\mathbf{e}_2$ = pure green, $\mathbf{e}_3$ = pure blue; red+green=yellow, red+blue=magenta, green+blue=cyan.
- The zero vector $\mathbf{0}$ is black; the all-ones column vector $\mathbf{1}$ is white — exactly the diagonal of the unit cube.
- Any 3D vector $\mathbf{x} = x_1\mathbf{e}_1 + x_2\mathbf{e}_2 + x_3\mathbf{e}_3$ — vector decomposition extended to three dimensions.
- Superscripts distinguish dimension where needed: $\mathbf{e}_1^{(2)}$, $\mathbf{e}_1^{(3)}$ denote $\mathbf{e}_1$ in 2D versus 3D.



Vector Length

$$\|\mathbf{x}\| = \|\mathbf{x}\|_2 = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}$$

- The Pythagorean theorem generalizes directly to the magnitude (length) of an n-dimensional vector $\mathbf{x}$.
- Also called the L^2 norm, Euclidean norm, or magnitude; the subscript 2 is often dropped.
- Mind the notation: absolute value uses $|-5|$, while vector magnitude uses $\|\mathbf{x}\|$.
- Example: $\mathbf{x} = [3, 4]^T$ gives $\|\mathbf{x}\| = \sqrt{3^2+4^2} = 5$; $[1, 2, 2]^T$ has length 3.

Computing Vector Magnitude

Code 2. Computing Vector Size |
LA_01_02_03.ipynb

- a `np.array()` defines two 1D arrays, a and b
- b / d `np.linalg.norm()` computes the length of a vector (1D array) directly
- c / e Manual check: index each component `a[i]`, square with `**2`, sum, then take `np.sqrt()`

a

b

c

d

e

```
## 初始化
import numpy as np

## 定义向量 (一维数组)
a = np.array([3, 4])
b = np.array([1, 2, 2])

## 计算向量长度 (L2范数)
length_a = np.linalg.norm(a)
np.sqrt(a[0]**2 + a[1]**2)

length_b = np.linalg.norm(b)
np.sqrt(b[0]**2 + b[1]**2 + b[2]**2)
```



SUMMARY

Key Terms to Remember

Real line 实数数轴

Cartesian coordinate system 笛卡尔坐标系

numpy.meshgrid() 网格数据生成函数

Span 张成

Vector decomposition 向量分解

Euclidean space 欧几里得空间 ($\mathbb{R}$, $\mathbb{R}^2$, $\mathbb{R}^3$)

Origin 原点

Unit vector 单位向量

Right-hand rule 右手定则

Magnitude / L² norm 向量长度 / 欧几里得范数 / 模

KEEP EXPLORING

Companion Resources & Further Reading

Matplotlib · Quiver Plots

matplotlib.org/stable/gallery/images_contours_and_fields/quiver_simple_demo.html

Seaborn · scatterplot

seaborn.pydata.org/generated/seaborn.scatterplot.html

Matplotlib · 3D scatter

matplotlib.org/stable/gallery/mplot3d/scatter3d.html

Open-Source Code & Manuscript

All code, Jupyter notebooks, and PDF drafts: <https://github.com/Visualize-ML>

Complete These with DeepSeek / ChatGPT and Similar Tools

Q1 Review the axioms and properties of Euclidean geometry.

Q3 Explain what the eight octants of the 3D coordinate system are.

Q5 Have DeepSeek/ChatGPT annotate the `seaborn.scatterplot` example line by line, and learn to draw 2D scatter plots.

Q7 Use `numpy.linalg.norm()` to compute the length of $[2,4,4]$, $[2,3,6]$, $[1,4,8]$, and verify with the Pythagorean theorem.

Q9 Self-study how to index and slice a 3D array.

Q2 Explain what the four quadrants of the planar Cartesian system are.

Q4 Have DeepSeek/ChatGPT annotate the `matplotlib.quiver_simple_demo` example line by line, and learn to draw arrow plots.

Q6 Have DeepSeek/ChatGPT annotate the `matplotlib.scatter3d` example line by line, and learn to draw 3D scatter plots.

Q8 Self-study how to create a 3D array with `numpy.meshgrid()`.

Q10 Write a Python function that draws an arbitrary real number line and marks a given real number's position.

Complete These with DeepSeek / ChatGPT and Similar Tools

Q11 Use `matplotlib.pyplot.quiver()` to draw the vectors $[2,3]^T$ and $[-1,4]^T$ in the Cartesian plane.

Q13 Write Python code verifying that $\mathbf{x} = x_1\mathbf{e}_1 + x_2\mathbf{e}_2$ holds for an arbitrary 2D vector $\mathbf{x}$.

Q15 Use `mplot3d's Axes3D` to draw the x_1, x_2, x_3 axes of a 3D coordinate system.

Q17 Use `numpy.linalg.norm()` to compute the length of $[6,8]^T$, $[1,1,1]^T$, and $[2,2,2,2]^T$.

Q19 Use `matplotlib` to draw the basis vectors $\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3$ in RGB color space (like Fig. 9a).

Q12 Use `numpy.meshgrid()` to build a 2D grid over $x_1, x_2 \in [-3,3]$ with step 0.5, and plot it with `matplotlib`.

Q14 Use `matplotlib` to draw a regular scatter grid on the “red-green” plane (like Fig. 4b), with color fading over x_1, x_2 .

Q16 Use `numpy.meshgrid()` to build 3D grid data, and print its shape.

Q18 Generate a random 3D vector and compare `numpy.linalg.norm()` against a manual $\sqrt{x_1^2 + x_2^2 + x_3^2}$ computation.

Q20 Write Python code that determines which quadrant a given 2D point (x_1, x_2) falls in.

Extended Practice Combined with Python

Q21 Write Python code that determines which octant a given 3D point (x_1, x_2, x_3) falls in.

Q23 Use Python to draw a unit square and mark its diagonal (the all-ones vector $\mathbf{1}$).

Q25 Compare the results of `numpy.meshgrid()`'s default 'xy' indexing versus matrix 'ij' indexing.

Q27 Use `seaborn.scatterplot()` to plot several random 2D vectors, colored by vector length.

Q29 Use `matplotlib` to draw the 1D line, 2D plane, and 3D space coordinate systems together, as a summary of this section.

Q22 Use `numpy` to generate scatter data for the RGB color cube, and visualize it with a 3D scatter plot (like Fig. 9b).

Q24 Use `numpy.cross()` to compute $\mathbf{e}_1 \times \mathbf{e}_2$, and verify it equals $\mathbf{e}_3$ — a concrete check of the right-hand rule.

Q26 Decompose the vector $[5, 12]^T$ into $x_1\mathbf{e}_1 + x_2\mathbf{e}_2$, and use `quiver()` to plot both components and the sum.

Q28 Write a function that takes an n-dimensional vector and returns its L2 norm; verify it against `numpy.linalg.norm()`.

10 Prompts to Explore with AI

- 1 Explain why the real line is also called “one-dimensional Euclidean space,” and how it differs from an ordinary number line.
- 2 Use a real-world example to explain the significance of the Cartesian coordinate system, and tell the story of how Descartes invented it.
- 3 Explain in detail how `numpy.meshgrid()` works, with an example illustrating what the outputs `xx1` and `xx2` mean.
- 4 Why does the grid from `numpy.meshgrid()` have negative values on top and positive values at the bottom vertically — and why is this the opposite of the usual math convention?
- 5 Explain the concept of “span,” and why $\mathbf{e}_1$ and $\mathbf{e}_2$ span the entire 2D plane.

10 Prompts to Explore with AI

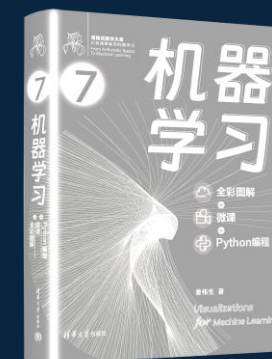
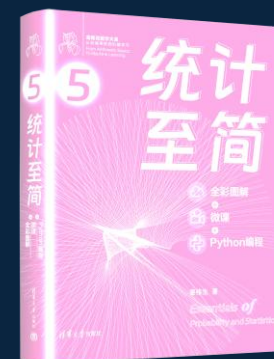
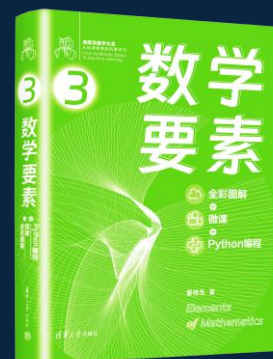
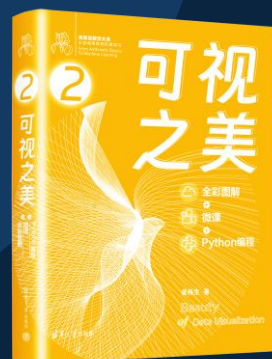
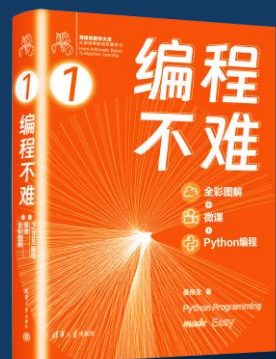
- 6 Explain the physical meaning of the right-hand rule, and how it applies to determinants, cross products, and rotation direction.
- 7 Explain how the RGB color space differs from a true vector space, and why it is not strictly a vector space.
- 8 Derive the formula for the length (L^2 norm) of an n -dimensional vector, and explain its connection to the Pythagorean theorem.
- 9 Compare the absolute-value notation $|x|$ with the vector-magnitude notation $\|\mathbf{x}\|$, explaining when each applies.
- 10 Design a small Python experiment that visually demonstrates how a 2D vector decomposes into components along the coordinate axes.

数学

数学基础

线性代数

概率统计



Python编程

数据可视化

工具

数据分析
图和网络

回归、降维
分类、聚类

实践

